

Source Patches — Modified Stock Open WebUI Files

The RegOS installer (`regos-installer/source-patches/apply-patches.py`) applies 15 surgical patches to stock Open WebUI source files. It also copies 3 new files into the source tree. This document is the authoritative record of every modification.

Patching method: Anchor-based injection (find text → inject before/after/replace). Idempotent — skips if already applied. Never replaces entire files.

New Files Injected (3)

File	Purpose
<code>src/lib/components/RegOSDisclaimerModal.svelte</code>	Modal that shows customizable disclaimer/service agreement on first login. Markdown-rendered body, configurable title + accept label.
<code>src/lib/components/admin/Settings/RegOS.svelte</code>	Admin settings panel for 3 RegOS feature groups: Disclaimer, Guest Access, Confidence Display.
<code>src/routes/(app)/admin/regos/page.svelte</code>	Admin page that hosts the RegOS settings component.

Backend Patches (7 files, 8 patches)

1. `backend/open_webui/config.py`

Patch: `patch_backend_config`

What changes:

- Adds `GUEST_MESSAGE_LIMIT` (default 10) and `GUEST_MESSAGE_WINDOW` (default 10800s) env vars
- Appends 13 `PersistentConfig` entries for RegOS settings: disclaimer (enabled, title, body, accept_label), guest (enabled, message_limit, generation_limit, session_ttl, show_button), confidence (enabled, style, high_threshold, medium_threshold)

2. `backend/open_webui/main.py`

Patch: `patch_backend_main`

What changes:

- Adds imports for all 13 RegOS config settings
- Assigns config values to `app.state.config` for runtime access
- Injects guest rate limiting into `chat_completion` endpoint: checks if user role is "guest", enforces chat count limit (`GUEST_MESSAGE_LIMIT`) and generation count limit (`REGOS_GUEST_GENERATION_LIMIT`), returns HTTP 429 when exceeded

3. backend/open_webui/routers/configs.py

Patch: patch_backend_configs_router

What changes:

- Adds 4 Pydantic models: RegOSDisclaimerForm, RegOSGuestForm, RegOSConfidenceForm, RegOSSettingsForm
- Adds 4 REST endpoints:
- GET /api/v1/configs/regos — admin-only, returns all RegOS settings
- POST /api/v1/configs/regos — admin-only, updates all RegOS settings
- GET /api/v1/configs/regos/public — authenticated, returns limited public settings
- GET /api/v1/configs/regos/guest-status — unauthenticated, returns guest enabled/show_button flags

4. backend/open_webui/routers/auths.py

Patch: patch_guest_endpoint

What changes:

- Replaces stock guest endpoint entirely (uses fragment file fragments/guest_endpoint.py)
- New endpoint: POST /api/v1/auths/guest with GuestSignInForm containing email: str
- Email validation, IP-based rate limiting, guest account reuse (if email exists), auto-assignment to "guest" group, JWT with configurable TTL, locked-down permissions

5. backend/open_webui/routers/users.py

Patch: patch_users_api_roles

What changes:

- Adds optional roles query parameter to GET /api/v1/users
- Enables filtering users by role(s) — comma-separated list

6. backend/open_webui/utils/models.py

Patch: patch_model_access_control

What changes:

- Model access control check: `user.role == "user"` → `user.role in ("user", "guest")`
- Ensures guests are subject to the same model access restrictions as regular users

7. backend/requirements.txt

Patch: patch_requirements

What changes:

- Adds neo4j Python driver package (required for GraphRAG Neo4j connectivity)

Frontend Patches (6 files, 7 patches)

8. `src/lib/apis/configs/index.ts`

Patch: `patch_frontend_configs_api`

What changes:

- Adds 4 TypeScript API functions: `getRegosConfig(token)`, `updateRegosConfig(token, config)`, `getRegosGuestStatus()`, `getRegosPublicConfig(token)`

9. `src/lib/apis/auths/index.ts`

Patch: `patch_guest_api_frontend`

What changes:

- Updates `userGuestSignIn` signature: `async ()` → `async (email: string)`
- Adds email to fetch request body

10. `src/lib/apis/users/index.ts`

Patch: `patch_users_frontend_api`

What changes:

- Adds optional `roles?: string` parameter to `getUsers()` function
- Includes roles in `searchParams` when provided

11. `src/routes/(app)/+layout.svelte`

Patch: `patch_app_layout`

What changes:

- Imports `RegOSDisclaimerModal` component and `getRegosPublicConfig` API function
- Adds `regosConfig` variable and `showRegOSDisclaimer` Svelte store
- `onMount`: fetches RegOS public config, shows disclaimer modal if enabled and not yet acknowledged (localStorage check)
- Renders `<RegOSDisclaimerModal>` in template

12. `src/routes/(app)/admin/+layout.svelte`

Patch: `patch_admin_layout`

What changes:

- Adds "RegOS" navigation tab in admin sidebar (routes to `/admin/regos`)

13. `src/routes/auth/+page.svelte`

Patch: `patch_auth_page`

What changes:

- Imports getRegosGuestStatus API function
- Adds guestEnabled, showGuestEmailInput, guestEmail reactive variables
- Replaces guest sign-in handler: first click shows email input, second click validates email and submits to /api/v1/auths/guest
- onMount: fetches guest status, conditionally shows guest button
- Replaces guest button template with conditional email input form

14. src/lib/components/admin/Users.svelte

Patch: patch_users_admin_guests_tab

What changes:

- Adds 'guests' to valid tabs: ['overview', 'guests', 'groups']
- Adds Guests tab button with icon
- Overview tab excludes guests (excludeRoles={['guest']}), Guests tab filters to guests only (filterRole="guest")

15. src/lib/components/admin/Users/UserList.svelte

Patch: patch_userlist_role_filter

What changes:

- Adds filterRole and excludeRoles export props
- Adds reactive rolesParam that builds query string for role filtering
- Updates getUsers() call to pass rolesParam

Installation Sequence

Step	Script	Action
1	01-detect-container.sh	Find and verify Open WebUI Docker container
2	02-install-deps.sh	Install neo4j Python driver in container
3	03-copy-data.sh	Copy JSON data files to container
4	04-copy-scripts.sh	Copy demo/verify scripts to container
5	05-register-functions.sh	Register graphrag_filter + audit_logger via Open WebUI API
6	06-create-model.sh	Create "RegOS Compliance Copilot" model with system prompt
7	07-setup-groups.sh	Optionally create "RegOS Testers" group

Step	Script	Action
8	08-verify.sh	Verify all components deployed correctly
9	09-guest-disclaimer.sh	Verify guest access + disclaimer system
10	10-regos-admin.sh	Verify RegOS admin panel endpoints

Note: Source patches (apply-patches.py) run separately — they modify the Open WebUI source before the Docker image is rebuilt. The install steps above operate on the running container.

Middleware Status

backend/open_webui/utils/middleware.py was modified 6 times during development (attempts at data transport between filters) and **fully rolled back to stock**. It is unmodified.